

REPORTE UNIFICADO DE CAMBIOS

Archivo eliminado: src/main/java/com/mantenimiento/azul/checker/ParenthesesChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class ParenthesesChecker extends Checker {
8 |
9 |     public ParenthesesChecker() {
10 |    }
11 |
12 |    @Override
13 |    public void pass(List<String> codeStream) throws InvalidLineFormatException {
14 |
15 |        boolean pass = codeStream.stream().anyMatch(line ->
16 |            Regex.ENDS_WITH_PARENTHESES.matcher(line).matches());
17 |
18 |        if (pass) {
19 |            throw new InvalidLineFormatException("No se permite terminar la línea de código con
20 |            paréntesis");
21 |        } else if (this.next != null) {
22 |            this.next.pass(codeStream);
23 |        }
24 |    }
25 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/model/FileStats.java

```
1 | package com.mantenimiento.azul.model;
2 |
```

```
3 | import java.nio.file.Paths;
4 | import java.util.ArrayList;
5 |
6 | public record FileStats (String fileName, int physicalLines, int lines, ArrayList<ClassCounter>
classes) {
7 |     public String fileName() {
8 |         return Paths.get(fileName).getFileName().toString();
9 |     }
10 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/processor/CodeProcessor.java

```
1 | package com.mantenimiento.azul.processor;
2 |
3 | import com.mantenimiento.azul.checker.Checker;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.model.FileStats;
6 | import com.mantenimiento.azul.utils.CodeLineAnalyzer;
7 | import java.io.IOException;
8 | import java.nio.file.Files;
9 | import java.nio.file.Path;
10 | import java.util.List;
11 |
12 | public class CodeProcessor {
13 |
14 |     private final Checker checkerChain;
15 |
16 |     public CodeProcessor(Checker checkerChain) {
17 |         this.checkerChain = checkerChain;
18 |     }
19 | }
```

```
20 | public FileStats processFile(Path filePath) throws IOException, InvalidLineFormatException
    | {
21 |     List<String> codeLines = Files.readAllLines(filePath);
22 |
23 |     if (checkerChain != null) {
24 |         checkerChain.pass(codeLines);
25 |     }
26 |
27 |     return CodeLineAnalyzer.analyzeFile(filePath);
28 | }
29 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/utils/Regex.java

```
1 | package com.mantenimiento.azul.utils;
2 |
3 | import java.util.regex.Pattern;
4 |
5 | public class Regex {
6 |
7 |     public static final Pattern ONLY_LEFT_CURLY_BARCE = Pattern.compile("\\s*\\{\\s*$");
8 |     public static final Pattern ENDS_WITH_PARENTHESES = Pattern.compile("\\.*[\\(\\)]$");
9 |     public static final Pattern MULTI_INSTANCE =
    | Pattern.compile("\\s*\\w+\\s+\\w+\\s*,\\s*\\w+(\\s*=\\s*[^,;]+)?(\\s*,\\s*\\w+(\\s*=\\s*[^,;]+)?)*\\s*");
10 |     public static final Pattern ENDS_WITH_WORD = Pattern.compile("\\.*b\\w+\\b$");
11 |
12 |     public static final Pattern SINGLE_LINE_COMMENT = Pattern.compile("//.*");
13 |
14 |     public static final Pattern BLOCK_COMMENT_START = Pattern.compile("/\\*.*");
15 |     public static final Pattern BLOCK_COMMENT_CONTINUE = Pattern.compile("^\\s*\\*.*$");
    | // It still doesn't work well
```

```

16 |     public static final Pattern BLOCK_COMMENT_END = Pattern.compile(".*\\*/");
17 |
18 |     public static final Pattern ENDS_WITH_BREAK = Pattern.compile(".*;\\s*\\}.**");
19 |
20 |     public static final Pattern LOGICAL_LINE = Pattern.compile(".*[{}$]");
21 |
22 |     public static final Pattern CLASS_IDENTIFIER = Pattern.compile(".*\\bclass\\b\\s\\w+.*");
23 |
24 |     public static final Pattern METHOD_IDENTIFIER =
25 |
Pattern.compile("(@\\w+\\s*)?(public|private|protected)?\\s*(static\\s*)?(abstract\\s*)?\\w+\\s+\\w+\\s+\\s*\\([^\n)]*\\)\\s*(throws\\s+[\\w\\s,]+)?\\s*(\\{;);");
26 |
27 |     public static final Pattern RECORD_IDENTIFIER =
Pattern.compile(".*\\breCORD\\b\\s\\w+.*");
28 | }

```

Archivo eliminado: src/main/java/com/mantenimiento/azul/model/ClassCounter.java

```
1 | package com.mantenimiento.azul.model;
2 |
3 | public class ClassCounter {
4 |     private int physicalLOC = 0;
5 |     private String name;
6 |     private int methodCount = 0;
7 |     private int lines = 0;
8 |
9 |     public ClassCounter(String name) {
10 |         this.name = name;
11 |     }
12 |
```

```
13 | public int getPhysicalLOC() {
14 |     return physicalLOC - 2; // -2 for the class declarations
15 | }
16 |
17 | public void addPhysicalLOC() {
18 |     this.physicalLOC++;
19 | }
20 |
21 | public String getName() {
22 |     return name;
23 | }
24 |
25 | public void incrementMethodCount() {
26 |     this.methodCount++;
27 | }
28 |
29 | public int getMethodCount() {
30 |     return methodCount;
31 | }
32 |
33 | public int getLines() {
34 |     return lines;
35 | }
36 |
37 | public void setLines(int lines) {
38 |     this.lines = lines;
39 | }
40 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/checker/MultiInstanceChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class MultiInstanceChecker extends Checker {
8 |
9 |     @Override
10 |     public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |         boolean pass = codeStream.stream().anyMatch(line ->
12 |             Regex.MULTI_INSTANCE.matcher(line).matches());
13 |         if (pass) {
14 |             throw new InvalidLineFormatException("No se permite instanciar varias variables en
15 |             una sola línea");
16 |         } else if (this.next != null) {
17 |             this.next.pass(codeStream);
18 |         }
19 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/utils/CodeLineAnalyzer.java

```
1 | package com.mantenimiento.azul.utils;
2 |
3 | import java.io.*;
4 | import java.nio.file.Path;
5 | import java.util.ArrayList;
6 | import java.util.Arrays;
```

```
7 |
8 | import com.mantenimiento.azul.model.*;
9 |
10 | public class CodeLineAnalyzer {
11 |
12 |     public static FileStats analyzeFile(Path filePath) {
13 |         int physicalLines = 0;
14 |         //int logicalLines = 0;
15 |         boolean insideBlockComment = false;
16 |         String currentClassName;
17 |         ArrayList<ClassCounter> classCounter = new ArrayList<ClassCounter>();
18 |         ClassCounter currentClass = null;
19 |         int lines = 0;
20 |
21 |         try (BufferedReader reader = new BufferedReader(new FileReader(filePath.toFile()))) {
22 |             String line;
23 |             while ((line = reader.readLine()) != null) {
24 |                 line = line.trim();
25 |
26 |                 // ----- Lines counter ----- //
27 |                 lines++;
28 |
29 |                 // ----- blank lines ----- //
30 |                 if (line.isEmpty()) continue;
31 |
32 |                 // ----- lines or blocks of comments ----- //
33 |                 if (insideBlockComment) {
34 |                     if (Regex.BLOCK_COMMENT_END.matcher(line).matches()) {
35 |                         insideBlockComment = false;
```

```
36 |         }
37 |         continue;
38 |     }
39 |     if (Regex.BLOCK_COMMENT_START.matcher(line).matches()) {
40 |         insideBlockComment = true;
41 |         continue;
42 |     }
43 |     if (Regex.BLOCK_COMMENT_CONTINUE.matcher(line).matches()) {
44 |         continue;
45 |     }
46 |
47 |     if (Regex.SINGLE_LINE_COMMENT.matcher(line).matches()) continue;
48 |
49 |     // ----- Class identification ----- //
50 |     if (Regex.CLASS_IDENTIFIER.matcher(line).matches()) {
51 |         String[] classLine = line.split(" ");
52 |         int indexClassWord = Arrays.asList(classLine).indexOf("class");
53 |         currentClassName = classLine[indexClassWord + 1];
54 |         currentClass = new ClassCounter(currentClassName);
55 |         classCounter.add(currentClass);
56 |     }
57 |
58 |     // ----- record identification ----- //
59 |     if (Regex.RECORD_IDENTIFIER.matcher(line).matches()) {
60 |         String[] recordLine = line.split(" ");
61 |         int indexClassWord = Arrays.asList(recordLine).indexOf("record");
62 |         currentClassName = recordLine[indexClassWord + 1];
63 |         currentClass = new ClassCounter(currentClassName);
64 |         classCounter.add(currentClass);
```



```
65 |     }
66 |
67 |     if (currentClass != null) {
68 |         currentClass.addPhysicalLOC();
69 |     }
70 |
71 |     // ----- Methods counter ----- //
72 |     if (currentClass != null && Regex.METHOD_IDENTIFIER.matcher(line).matches()) {
73 |         currentClass.incrementMethodCount();
74 |     }
75 |
76 |     if(line.contains("else") || line.contains("catch") || line.contains("finally")){
77 |         continue;
78 |     }
79 |
80 |     // ----- Logical lines counter ----- //
81 |     /*
82 |     if (Regex.LOGICAL_LINE.matcher(line).matches()) {
83 |         logicalLines++;
84 |     }
85 |     */
86 |     // ----- Physical line counter ----- //
87 |     physicalLines++;
88 | }
89 |
90 | if (currentClass!=null) {
91 |     currentClass.setLines(lines);
92 | }
93 | } catch (IOException e) {
```

```
94 |         System.err.println("Error al leer el archivo: " + filePath);
95 |         return null;
96 |     }
97 |
98 |     return new FileStats(filePath.toString(), physicalLines, lines, classCounter);
99 | }
100 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/Main.java

```
1 | package com.mantenimiento.azul;
2 |
3 | import com.mantenimiento.azul.checker.MultiInstanceChecker;
4 | import com.mantenimiento.azul.checker.ParenthesesChecker;
5 | import com.mantenimiento.azul.checker.Checker;
6 | import com.mantenimiento.azul.checker.EndBreakChecker;
7 | import com.mantenimiento.azul.checker.LeftCurlyBraceChecker;
8 | import com.mantenimiento.azul.processor.CodeProcessor;
9 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
10 | import com.mantenimiento.azul.model.FileStats;
11 |
12 | import java.io.IOException;
13 | import java.nio.file.*;
14 | import java.nio.file.attribute.BasicFileAttributes;
15 | import java.util.ArrayList;
16 | import java.util.List;
17 | import java.util.Scanner;
18 |
19 | public class Main {
20 |
```

```
21 | public static void main(String[] args) {
22 |     Scanner scanner = new Scanner(System.in);
23 |     String projectPath = args.length > 0 ? args[0] : ".";
24 |
25 |     while (true) {
26 |         System.out.println("");
27 |         System.out.print("Introduzca la ruta del proyecto:");
28 |         String inputPath = scanner.nextLine().trim();
29 |         System.out.println("");
30 |
31 |         if (!inputPath.isEmpty()) {
32 |             projectPath = inputPath;
33 |         }
34 |
35 |         Checker checkerChain = createCheckerChain();
36 |         CodeProcessor processor = new CodeProcessor(checkerChain);
37 |
38 |         List<FileStats> results = analyzeProject(projectPath, processor);
39 |         if (!results.isEmpty()) {
40 |             printResults(results, projectPath);
41 |         }
42 |
43 |         System.out.println("");
44 |         System.out.print("¿Desea analizar otra ruta? (y/n): ");
45 |         String respuesta = scanner.nextLine().trim().toLowerCase();
46 |         if (!(respuesta.equals("y") || respuesta.equals("yes"))) {
47 |             System.out.println("");
48 |             break;
49 |         }
```

```
50 |     }
51 |
52 |     scanner.close();
53 | }
54 |
55 | private static Checker createCheckerChain() {
56 |     ParenthesesChecker parenthesesChecker = new ParenthesesChecker();
57 |     LeftCurlyBraceChecker leftCurlyBraceChecker = new LeftCurlyBraceChecker();
58 |     MultiInstanceChecker multiInstanceChecker = new MultiInstanceChecker();
59 |     EndBreakChecker endBreakChecker = new EndBreakChecker();
60 |
61 |     parenthesesChecker.setNext(leftCurlyBraceChecker);
62 |     leftCurlyBraceChecker.setNext(multiInstanceChecker);
63 |     multiInstanceChecker.setNext(endBreakChecker);
64 |
65 |     return parenthesesChecker;
66 | }
67 |
68 | private static List<FileStats> analyzeProject(String projectPath, CodeProcessor processor)
69 | {
70 |
71 |     try {
72 |         Files.walkFileTree(Paths.get(projectPath), new SimpleFileVisitor<Path>() {
73 |             @Override
74 |             public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
75 |                 if (file.toString().endsWith(".java")) {
76 |                     processFile(file, processor, results);
77 |                 }
78 |                 return FileVisitResult.CONTINUE;
79 |             }
80 |         });
81 |     } catch (IOException e) {
82 |         throw new RuntimeException(e);
83 |     }
84 | }
```

```
79 |     }
80 |     });
81 | } catch (IOException e) {
82 |     System.err.println("Error al analizar el proyecto: " + e.getMessage());
83 | }
84 |
85 |     return results;
86 | }
87 |
88 | private static void processFile(Path file, CodeProcessor processor, List<FileStats> results) {
89 |     try {
90 |         FileStats stats = processor.processFile(file);
91 |         results.add(stats);
92 |     } catch (InvalidLineFormatException e) {
93 |         System.err.println("Error en " + file + ": " + e.getMessage());
94 |     } catch (IOException e) {
95 |         System.err.println("No se pudo leer " + file);
96 |     }
97 | }
98 |
99 | private static void printResults(List<FileStats> results, String projectPath) {
100 |     int totalPhysicalLines = 0;
101 |     //int totalLogicalLines = 0; //It was discontinued due to changing requirements
102 |     int totalLines = 0;
103 |
104 |     String[] projectP = projectPath.split("\\\\");
105 |     String projectName = projectP[projectP.length - 1];
106 |
107 |     System.out.println("");
```

```
108 |     System.out.println("Programa: " + projectName);
109 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",
110 |         "Clase", "Metodos", "LOC f Clase", "Lineas Clase", "LOC f Programa", "Lineas
Programa");
111 |     System.out.println("");
112 |     System.out.println("=".repeat(117));
113 |
114 |     for (FileStats stats : results) {
115 |
116 |         totalPhysicalLines += stats.physicalLines();
117 |         //totalLogicalLines += stats.logicalLines();
118 |         totalLines += stats.lines();
119 |
120 |         for(int i = 0; i < stats.classes().size(); i++) {
121 |             System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",
122 |                 stats.classes().get(i).getName(),
123 |                 stats.classes().get(i).getMethodCount(),
124 |                 stats.classes().get(i).getPhysicalLOC(),
125 |                 stats.classes().get(i).getLines(),
126 |                 "",
127 |                 "");
128 |             System.out.println("");
129 |         }
130 |     }
131 |
132 |     System.out.println("=".repeat(117));
133 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",
134 |         "", "", "", "", totalPhysicalLines, totalLines);
135 | }
136 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/checker/LeftCurlyBraceChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class LeftCurlyBraceChecker extends Checker {
8 |
9 |     @Override
10 |     public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |
12 |         boolean pass = codeStream.stream().anyMatch(line ->
13 |             Regex.ONLY_LEFT_CURLY_BARCE.matcher(line).matches());
14 |
15 |         if (pass) {
16 |             throw new InvalidLineFormatException("No se permite tener la llave abierta como
17 |             único elemento de la línea");
18 |         } else if (this.next != null) {
19 |             this.next.pass(codeStream);
20 |         }
21 |     }
22 | }
```

Archivo eliminado:

src/main/java/com/mantenimiento/azul/exception/InvalidLineFormatException.java

```
1 | package com.mantenimiento.azul.exception;
2 |
3 | public class InvalidLineFormatException extends Exception{
```

```
4 |     public InvalidLineFormatException (String message) {
5 |         super(message);
6 |     }
7 | }
```

Archivo eliminado: src/main/java/com/mantenimiento/azul/checker/EndBreakChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class EndBreakChecker extends Checker {
8 |
9 |     @Override
10 |     public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |
12 |         boolean pass = codeStream.stream().
13 |             anyMatch(line -> Regex.ENDS_WITH_BREAK.matcher(line).matches() &&
14 |                 !Regex.BLOCK_COMMENT_START.matcher(line).matches());
15 |
16 |         if (pass) {
17 |             throw new InvalidLineFormatException("No se sigue la estructura de cierre con llave");
18 |         } else if (this.next != null) {
19 |             this.next.pass(codeStream);
20 |         }
21 |     }
22 |
23 | }
```


Archivo eliminado: src/main/java/com/mantenimiento/azul/checker/Checker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 |
6 | public abstract class Checker {
7 |     protected Checker next;
8 |
9 |     public void setNext(Checker checker) {
10 |         this.next = checker;
11 |     }
12 |
13 |     public abstract void pass(List<String> codeList) throws InvalidLineFormatException;
14 | }
```

**Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/checker/CheckerFactory.java**

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | public class CheckerFactory {
4 |     public static Checker createCheckerChain() {
5 |         Checker parenthesesChecker = new ParenthesesChecker();
6 |         Checker leftCurlyBraceChecker = new LeftCurlyBraceChecker();
7 |         Checker multiInstanceChecker = new MultiInstanceChecker();
8 |         Checker endBreakChecker = new EndBreakChecker();
9 |
10 |         parenthesesChecker.setNext(leftCurlyBraceChecker);
11 |         leftCurlyBraceChecker.setNext(multiInstanceChecker);
```

```
12 |     multiInstanceChecker.setNext(endBreakChecker);
13 |
14 |     return parenthesesChecker;
15 | }
16 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/checker/LeftCurlyBraceChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class LeftCurlyBraceChecker extends Checker {
8 |
9 |     @Override
10 |     public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |
12 |         boolean pass = codeStream.stream().anyMatch(line ->
13 |             Regex.ONLY_LEFT_CURLY_BARCE.matcher(line).matches());
14 |
15 |         if (pass) {
16 |             throw new InvalidLineFormatException("No se permite tener la llave abierta como
17 |             único elemento de la línea");
18 |         } else if (this.next != null) {
19 |             this.next.pass(codeStream);
20 |         }
21 |     }
22 | }
```

Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/Main.java

```
1 | package com.mantenimiento.azul;
2 |
3 | import com.mantenimiento.azul.checker.MultiInstanceChecker;
4 | import com.mantenimiento.azul.checker.ParenthesesChecker;
5 | import com.mantenimiento.azul.comparator.VersionComparator;
6 | import com.mantenimiento.azul.checker.Checker;
7 | import com.mantenimiento.azul.checker.EndBreakChecker;
8 | import com.mantenimiento.azul.checker.LeftCurlyBraceChecker;
9 | import com.mantenimiento.azul.processor.CodeProcessor;
10 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
11 | import com.mantenimiento.azul.model.FileStats;
12 |
13 | import java.io.IOException;
14 | import java.nio.file.*;
15 | import java.nio.file.attribute.BasicFileAttributes;
16 | import java.util.ArrayList;
17 | import java.util.HashSet;
18 | import java.util.List;
19 | import java.util.Scanner;
20 | import java.util.Set;
21 |
22 | public class Main {
23 |
24 |     public static void main(String[] args) {
25 |         Scanner scanner = new Scanner(System.in);
26 |         String projectPath = args.length > 0 ? args[0] : ".";
27 |
```

```
28 |     while (true) {
29 |         System.out.print("\nIntroduzca la ruta de la versión anterior: ");
30 |         String oldVersionPath = scanner.nextLine().trim();
31 |
32 |         System.out.print("\nIntroduzca la ruta de la nueva versión: ");
33 |         String newVersionPath = scanner.nextLine().trim();
34 |
35 |         Set<String> oldFiles = collectJavaFiles(oldVersionPath);
36 |         Set<String> newFiles = collectJavaFiles(newVersionPath);
37 |
38 |         VersionComparator.ComparisonResult result = VersionComparator.compare(oldFiles,
newFiles);
39 |
40 |         printComparisonResults(result);
41 |
42 |         Checker checkerChain = createCheckerChain();
43 |         CodeProcessor processor = new CodeProcessor(checkerChain);
44 |
45 |         List<FileStats> results = analyzeProject(projectPath, processor);
46 |         if (!results.isEmpty()) {
47 |             printResults(results, projectPath);
48 |         }
49 |
50 |         System.out.println("");
51 |         System.out.print("¿Desea analizar otra ruta? (y/n): ");
52 |         String respuesta = scanner.nextLine().trim().toLowerCase();
53 |         if (!(respuesta.equals("y") || respuesta.equals("yes"))) {
54 |             System.out.println("");
55 |             break;
56 |         }
```

```
57 |     }
58 |
59 |     scanner.close();
60 | }
61 |
62 | private static Set<String> collectJavaFiles(String projectPath) {
63 |     Set<String> javaFiles = new HashSet<>();
64 |     Path root = Paths.get(projectPath).normalize().toAbsolutePath();
65 |
66 |     try {
67 |         Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
68 |             @Override
69 |             public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
70 |                 if (file.toString().toLowerCase().endsWith(".java")) {
71 |                     // Convertir a ruta relativa como cadena con mayúsculas originales
72 |                     String relativePath = root.relativize(file).toString();
73 |                     javaFiles.add(relativePath);
74 |                 }
75 |                 return FileVisitResult.CONTINUE;
76 |             }
77 |         });
78 |     } catch (IOException e) {
79 |         System.err.println("Error: " + e.getMessage());
80 |     }
81 |     return javaFiles;
82 | }
83 |
84 | private static void printComparisonResults(VersionComparator.ComparisonResult result) {
85 |     System.out.println("\n=== Comparación de Versiones ===");
```

```
86 |     System.out.println("Archivos nuevos (" + result.addedFiles.size() + "):");
87 |     result.addedFiles.forEach(p -> System.out.println(" [A] " + p));
88 |
89 |     System.out.println("\nArchivos eliminados (" + result.removedFiles.size() + "):");
90 |     result.removedFiles.forEach(p -> System.out.println(" [D] " + p));
91 |
92 |     System.out.println("\nArchivos sin cambios (" + result.unchangedFiles.size() + "):");
93 |     result.unchangedFiles.forEach(p -> System.out.println(" [=] " + p));
94 | }
95 |
96 |
97 | private static Checker createCheckerChain() {
98 |     ParenthesesChecker parenthesesChecker = new ParenthesesChecker();
99 |     LeftCurlyBraceChecker leftCurlyBraceChecker = new LeftCurlyBraceChecker();
100 |     MultiInstanceChecker multiInstanceChecker = new MultiInstanceChecker();
101 |     EndBreakChecker endBreakChecker = new EndBreakChecker();
102 |
103 |     parenthesesChecker.setNext(leftCurlyBraceChecker);
104 |     leftCurlyBraceChecker.setNext(multiInstanceChecker);
105 |     multiInstanceChecker.setNext(endBreakChecker);
106 |
107 |     return parenthesesChecker;
108 | }
109 |
110 | private static List<FileStats> analyzeProject(String projectPath, CodeProcessor
processor) {
111 |     List<FileStats> results = new ArrayList<>();
112 |
113 |     try {
114 |         Files.walkFileTree(Paths.get(projectPath), new SimpleFileVisitor<Path>() {
```

```
115 |         @Override
116 |         public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
117 |             if (file.toString().endsWith(".java")) {
118 |                 processFile(file, processor, results);
119 |             }
120 |             return FileVisitResult.CONTINUE;
121 |         }
122 |     });
123 | } catch (IOException e) {
124 |     System.err.println("Error al analizar el proyecto: " + e.getMessage());
125 | }
126 |
127 | return results;
128 | }
129 |
130 | private static void processFile(Path file, CodeProcessor processor, List<FileStats> results)
131 | {
132 |     try {
133 |         FileStats stats = processor.processFile(file);
134 |         results.add(stats);
135 |     } catch (InvalidLineFormatException e) {
136 |         System.err.println("Error en " + file + ": " + e.getMessage());
137 |     } catch (IOException e) {
138 |         System.err.println("No se pudo leer " + file);
139 |     }
140 | }
141 |
142 | private static void printResults(List<FileStats> results, String projectPath) {
143 |     int totalPhysicalLines = 0;
144 |     //int totalLogicalLines = 0; //It was discontinued due to changing requirements
```

```
144 |     int totalLines = 0;
145 |
146 |     String[] projectP = projectPath.split("\\\\");
147 |     String projectName = projectP[projectP.length - 1];
148 |
149 |     System.out.println("");
150 |     System.out.println("Programa: " + projectName);
151 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",
152 |         "Clase", "Metodos", "LOC f Clase", "Lineas Clase", "LOC f Programa", "Lineas
Programa");
153 |     System.out.println("");
154 |     System.out.println("=".repeat(117));
155 |
156 |     for (FileStats stats : results) {
157 |
158 |         totalPhysicalLines += stats.physicalLines();
159 |         //totalLogicalLines += stats.logicalLines();
160 |         totalLines += stats.lines();
161 |
162 |         for(int i = 0; i < stats.classes().size(); i++) {
163 |             System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",
164 |                 stats.classes().get(i).getName(),
165 |                 stats.classes().get(i).getMethodCount(),
166 |                 stats.classes().get(i).getPhysicalLOC(),
167 |                 stats.classes().get(i).getLines(),
168 |                 "",
169 |                 "");
170 |             System.out.println("");
171 |         }
172 |     }
```



```
173 |  
174 |     System.out.println("=".repeat(117));  
175 |     System.out.printf("%-30s | %-10s | %-15s | %-15s | %-15s | %-15s |",  
176 |         "", "", "", "", totalPhysicalLines, totalLines);  
177 | }  
178 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/exception/InvalidLineFormatException.java

```
1 | package com.mantenimiento.azul.exception;  
2 |  
3 | public class InvalidLineFormatException extends Exception{  
4 |     public InvalidLineFormatException (String message) {  
5 |         super(message);  
6 |     }  
7 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/utils/Regex.java

```
1 | package com.mantenimiento.azul.utils;  
2 |  
3 | import java.util.regex.Pattern;  
4 |  
5 | public class Regex {  
6 |  
7 |     public static final Pattern ONLY_LEFT_CURLY_BARCE = Pattern.compile("\\s*\\{\\s*$");  
8 |     public static final Pattern ENDS_WITH_PARENTHESSES = Pattern.compile(".*[\\(\\)]$");  
9 |     public static final Pattern MULTI_INSTANCE =  
10 | Pattern.compile("\\s*\\w+\\s+\\w+\\s*,\\s*\\w+(\\s*=\\s*[^,;]+)?(\\s*,\\s*\\w+(\\s*=\\s*[^,;]+)?)*\\s*");  
11 |  
12 |     public static final Pattern ENDS_WITH_WORD = Pattern.compile(".*\\b\\w+\\b$");  
13 | }
```



```
7 | public class EndBreakChecker extends Checker {
8 |
9 |     @Override
10 |     public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |
12 |         boolean pass = codeStream.stream().
13 |             anyMatch(line -> Regex.ENDS_WITH_BREAK.matcher(line).matches() &&
14 |                 !Regex.BLOCK_COMMENT_START.matcher(line).matches());
15 |
16 |         if (pass) {
17 |             throw new InvalidLineFormatException("No se sigue la estructura de cierre con llave");
18 |         } else if (this.next != null) {
19 |             this.next.pass(codeStream);
20 |         }
21 |     }
22 |
23 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/checker/ParenthesesChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class ParenthesesChecker extends Checker {
8 |
9 |     public ParenthesesChecker() {
10 |     }
```

```
11 |
12 |  @Override
13 |  public void pass(List<String> codeStream) throws InvalidLineFormatException {
14 |
15 |      boolean pass = codeStream.stream().anyMatch(line ->
Regex.ENDS_WITH_PARENTHESSES.matcher(line).matches());
16 |      if (pass) {
17 |          throw new InvalidLineFormatException("No se permite terminar la línea de código con
paréntesis");
18 |      } else if (this.next != null) {
19 |          this.next.pass(codeStream);
20 |      }
21 |  }
22 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/checker/MultiInstanceChecker.java

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.utils.Regex;
6 |
7 | public class MultiInstanceChecker extends Checker {
8 |
9 |  @Override
10 |  public void pass(List<String> codeStream) throws InvalidLineFormatException {
11 |      boolean pass = codeStream.stream().anyMatch(line ->
Regex.MULTI_INSTANCE.matcher(line).matches());
12 |      if (pass) {
```

```
13 |         throw new InvalidLineFormatException("No se permite instanciar varias variables en  
una sola línea");  
14 |     } else if (this.next != null) {  
15 |         this.next.pass(codeStream);  
16 |     }  
17 | }  
18 |  
19 | }
```

**Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/model/FileStats.java**

```
1 | package com.mantenimiento.azul.model;  
2 |  
3 | import java.nio.file.Paths;  
4 | import java.util.ArrayList;  
5 |  
6 | public record FileStats (String fileName, int physicalLines, int lines, ArrayList<ClassCounter>  
classes) {  
7 |     public String fileName() {  
8 |         return Paths.get(fileName).getFileName().toString();  
9 |     }  
10 | }
```

**Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/utils/CodeLineAnalyzer.java**

```
1 | package com.mantenimiento.azul.utils;  
2 |  
3 | import java.io.*;  
4 | import java.nio.file.Path;  
5 | import java.util.ArrayList;
```

```
6 | import java.util.Arrays;
7 |
8 | import com.mantenimiento.azul.model.*;
9 |
10 | public class CodeLineAnalyzer {
11 |
12 |     public static FileStats analyzeFile(Path filePath) {
13 |         int physicalLines = 0;
14 |         //int logicalLines = 0;
15 |         boolean insideBlockComment = false;
16 |         String currentClassName;
17 |         ArrayList<ClassCounter> classCounter = new ArrayList<ClassCounter>();
18 |         ClassCounter currentClass = null;
19 |         int lines = 0;
20 |
21 |         try (BufferedReader reader = new BufferedReader(new FileReader(filePath.toFile()))) {
22 |             String line;
23 |             while ((line = reader.readLine()) != null) {
24 |                 line = line.trim();
25 |
26 |                 // ----- Lines counter ----- //
27 |                 lines++;
28 |
29 |                 // ----- blank lines ----- //
30 |                 if (line.isEmpty()) continue;
31 |
32 |                 // ----- lines or blocks of comments ----- //
33 |                 if (insideBlockComment) {
34 |                     if (Regex.BLOCK_COMMENT_END.matcher(line).matches()) {
```

```
35 |         insideBlockComment = false;
36 |     }
37 |     continue;
38 | }
39 | if (Regex.BLOCK_COMMENT_START.matcher(line).matches()) {
40 |     insideBlockComment = true;
41 |     continue;
42 | }
43 | if (Regex.BLOCK_COMMENT_CONTINUE.matcher(line).matches()) {
44 |     continue;
45 | }
46 |
47 | if (Regex.SINGLE_LINE_COMMENT.matcher(line).matches()) continue;
48 |
49 | // ----- Class identification ----- //
50 | if (Regex.CLASS_IDENTIFIER.matcher(line).matches()) {
51 |     String[] classLine = line.split(" ");
52 |     int indexClassWord = Arrays.asList(classLine).indexOf("class");
53 |     currentClassName = classLine[indexClassWord + 1];
54 |     currentClass = new ClassCounter(currentClassName);
55 |     classCounter.add(currentClass);
56 | }
57 |
58 | // ----- record identification ----- //
59 | if (Regex.RECORD_IDENTIFIER.matcher(line).matches()) {
60 |     String[] recordLine = line.split(" ");
61 |     int indexClassWord = Arrays.asList(recordLine).indexOf("record");
62 |     currentClassName = recordLine[indexClassWord + 1];
63 |     currentClass = new ClassCounter(currentClassName);
```

```
64 |         classCounter.add(currentClass);
65 |     }
66 |
67 |     if (currentClass != null) {
68 |         currentClass.addPhysicalLOC();
69 |     }
70 |
71 |     // ----- Methods counter ----- //
72 |     if (currentClass != null && Regex.METHOD_IDENTIFIER.matcher(line).matches()) {
73 |         currentClass.incrementMethodCount();
74 |     }
75 |
76 |     if(line.contains("else") || line.contains("catch") || line.contains("finally")){
77 |         continue;
78 |     }
79 |
80 |     // ----- Logical lines counter ----- //
81 |     /*
82 |     if (Regex.LOGICAL_LINE.matcher(line).matches()) {
83 |         logicalLines++;
84 |     }
85 |     */
86 |     // ----- Physical line counter ----- //
87 |     physicalLines++;
88 | }
89 |
90 | if (currentClass!=null) {
91 |     currentClass.setLines(lines);
92 | }
```



```
93 |         } catch (IOException e) {
94 |             System.err.println("Error al leer el archivo: " + filePath);
95 |             return null;
96 |         }
97 |
98 |         return new FileStats(filePath.toString(), physicalLines, lines, classCounter);
99 |     }
100 | }
```

**Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/processor/CodeProcessor.java**

```
1 | package com.mantenimiento.azul.processor;
2 |
3 | import com.mantenimiento.azul.checker.Checker;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 | import com.mantenimiento.azul.model.FileStats;
6 | import com.mantenimiento.azul.utils.CodeLineAnalyzer;
7 | import java.io.IOException;
8 | import java.nio.file.Files;
9 | import java.nio.file.Path;
10 | import java.util.List;
11 |
12 | public class CodeProcessor {
13 |
14 |     private final Checker checkerChain;
15 |
16 |     public CodeProcessor(Checker checkerChain) {
17 |         this.checkerChain = checkerChain;
18 |     }
19 | }
```

```
20 | public FileStats processFile(Path filePath) throws IOException, InvalidLineFormatException
    | {
21 |     List<String> codeLines = Files.readAllLines(filePath);
22 |
23 |     if (checkerChain != null) {
24 |         checkerChain.pass(codeLines);
25 |     }
26 |
27 |     return CodeLineAnalyzer.analyzeFile(filePath);
28 | }
29 | }
```

Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/model/ClassCounter.java

```
1 | package com.mantenimiento.azul.model;
2 |
3 | public class ClassCounter {
4 |     private int physicalLOC = 0;
5 |     private String name;
6 |     private int methodCount = 0;
7 |     private int lines = 0;
8 |
9 |     public ClassCounter(String name) {
10 |         this.name = name;
11 |     }
12 |
13 |     public int getPhysicalLOC() {
14 |         return physicalLOC - 2; // -2 for the class declarations
15 |     }
16 | }
```

```
17 | public void addPhysicalLOC() {
18 |     this.physicalLOC++;
19 | }
20 |
21 | public String getName() {
22 |     return name;
23 | }
24 |
25 | public void incrementMethodCount() {
26 |     this.methodCount++;
27 | }
28 |
29 | public int getMethodCount() {
30 |     return methodCount;
31 | }
32 |
33 | public int getLines() {
34 |     return lines;
35 | }
36 |
37 | public void setLines(int lines) {
38 |     this.lines = lines;
39 | }
40 | }
```

Archivo añadido: Contador-Cambios-Amarillo-

main/src/main/java/com/mantenimiento/azul/comparator/VersionComparator.java

```
1 | package com.mantenimiento.azul.comparator;
2 |
3 | import java.util.HashSet;
```

```
4 | import java.util.Set;
5 |
6 | public class VersionComparator {
7 |
8 |     public static class ComparisonResult {
9 |         public Set<String> addedFiles = new HashSet<>();
10 |         public Set<String> removedFiles = new HashSet<>();
11 |         public Set<String> unchangedFiles = new HashSet<>();
12 |     }
13 |
14 |     public static ComparisonResult compare(Set<String> version1, Set<String> version2) {
15 |         ComparisonResult result = new ComparisonResult();
16 |
17 |         // Convertir a rutas estilo UNIX para consistencia
18 |         Set<String> v1 = normalizePaths(version1);
19 |         Set<String> v2 = normalizePaths(version2);
20 |
21 |         result.removedFiles.addAll(v1);
22 |         result.removedFiles.removeAll(v2);
23 |
24 |         result.addedFiles.addAll(v2);
25 |         result.addedFiles.removeAll(v1);
26 |
27 |         result.unchangedFiles.addAll(v1);
28 |         result.unchangedFiles.retainAll(v2);
29 |
30 |         return result;
31 |     }
32 |
```

```
33 | private static Set<String> normalizePaths(Set<String> paths) {
34 |     Set<String> normalized = new HashSet<>();
35 |     for (String path : paths) {
36 |         // Mantener mayúsculas pero unificar separadores
37 |         normalized.add(path.replace("\\", "/"));
38 |     }
39 |     return normalized;
40 | }
41 | }
```

**Archivo añadido: Contador-Cambios-Amarillo-
main/src/main/java/com/mantenimiento/azul/checker/Checker.java**

```
1 | package com.mantenimiento.azul.checker;
2 |
3 | import java.util.List;
4 | import com.mantenimiento.azul.exception.InvalidLineFormatException;
5 |
6 | public abstract class Checker {
7 |     protected Checker next;
8 |
9 |     public void setNext(Checker checker) {
10 |         this.next = checker;
11 |     }
12 |
13 |     public abstract void pass(List<String> codeList) throws InvalidLineFormatException;
14 | }
```